Non-Functional Requirements (NFR) Benchmarking for IT Automation Agents

COMS6998 - IBM Mentor Project

Team Members

Arjun Vaidya (av3315)
Ayush Bhauwala (ab6106)
Gautam Agarwal (ga2726)
Rishabh Jain (rj2790)
Tanmay Agrawal (ta2832)

IBM Mentor

Ruchi Mahindru
Distinguished Engineer, IBM Research

Motivation: Gap in AI Agent Evaluation

Benchmarks ≠ Production Readiness

ITBench and AgentBench, primarily focus on evaluating task success.

Real-world IT environments prioritize how a task is completed not merely its successful outcome.

No NFR Framework for Agents

Software quality (ISO/IEC 25010) and ML (ISO/IEC 25059, ISO/IEC 5338) standards exist.

Framework for NFRs for AI agents remains absent.

IT Automation is NFR-Critical

ITBench targets critical domains like SRE, CISO, and FinOps. I

NFR failures can lead to significant operational disruptions and substantial costs.

Takeaway: Without NFR-aware evaluation, benchmarks do not translate to deployable agents.

Problem Definition

Lack of Domain-Specific NFR Understanding

A critical gap exists in defining which NFRs are paramount for specific IT domains (SRE, CISO, FinOps).

There is no clear view of agent-specific NFR priorities in existing literature and practical application.

Contribution: We developed a comprehensive survey paper analyzing NFRs related to IT Automation Agents.

Absence of Practical NFR Measurement in Benchmarks

Current benchmarks, including ITBench, exclusively evaluate task success.

NFRs such as latency, cost, reliability, are neither adequately instrumented nor systematically scored.

Contribution: We extended ITBench with an NFR-aware evaluation framework.

Level 1 Category	Level 2 Metric
Performance	End-to-end latency
	Planning overhead
	Tool call latency
	Time-to-first-token
	Throughput under load
	Workflow path efficiency
	Context window utilization
Cost Efficiency	Token consumption per task
	API call count
	Cost per successful/failed task
	Retry-induced overhead
Reliability	Task success rate
	Error rate by type
	Retry frequency
	Idempotency
	Regression resistance
Explainability	Trace completeness
	Decision explainability
	Workflow milestone visibility
	Guardrail/policy adherence visibility
	OpenTelemetry compatibility
Scalability	Concurrency capacity
	Multi-agent coordination efficiency
	Cross-domain scaling
Security	Prompt injection resilience
	Access control adherence
	Data sanitization
	Auditability
Usability	Human intervention frequency
	Time-to-correct
	Output interpretability

Technical Approach: Building an NFR Taxonomy

Synthesized Findings

We meticulously synthesized insights from established international standards:

- ISO/IEC 25010 for software quality.
- ISO/IEC 25059 for AI system quality.

Proposed Two-Level Taxonomy

Based on this synthesis, we proposed a novel two-level NFR taxonomy. This taxonomy is specifically designed to categorize and evaluate the non-functional attributes critical for the performance and deployability of AI agents in IT automation.

Categorization of NFR Metrics

Performance

- End-to-End Latency
- Tool Call Latency
- Planning Overhead
- Time To First Token (TTFT)
- Token Generation per Second

Cost Efficiency

- Total Cost
- LLM Call Volume
- Token Usage per Task
- Token Types
- Prompt Completion Ratio
- Tool Reuse Rate

Reliability

- Error Rate
- Context Window Utilization

NFRs Measured via Langfuse Trace

NFR	Formula
End-to-End Latency	`root_span.end_time - root_span.start_time`
Tool Call Latency	`obs.latency where obs.type == "TOOL"`
Planning Overhead	`(Σ reasoning_tokens / Σ output_tokens) * 100`
LLM Call Volume	`COUNT(obs) (obs.model is not None)`
Time To First Token (TTFT)	`obs.completion_start_time - obs.llm_call`
Token Generation per Second	`(obs.completion_end_time - obs.completion_start_time)/ output_tokens`
Token Usage per Task	`AVG(usage_details["total"]) grouped by task_id`
Tool Round Trip Time	`obs.tool_end_time - obs.tool_start_time`
Prompt Completion Ratio	`num input tokens / num output tokens`
Context Window Utilization	`total_tokens / context_window_size * 100 %`
Error Rate	`Total Errors / Total Operations × 100 %`
Tool Reuse Rate	`COUNT(obs.name="tool repeated usage") / COUNT(obs.type="TOOL")`

NFRs for Stream APIs

NFR	Formula
Time to First Token (TTFT)	`obs.completion_start_time - obs.llm_call`
Token Generation Speed	`(obs.completion_end_time - obs.completion_start_time)/token_count`

SRE and CISO do not inherently support streaming mode
Changed Agent tools to streaming mode to measure TTFT and tokens/second

vLLM for Fine-Grained LLM Metrics

Langfuse/CrewAI does not provide granular LLM and system level metrics.
Implemented sandbox benchmarking framework for CISO Kubernetes, Kyverno Agents

- **Max KV Cache & Context Window Utilization %:** Insight into memory stress of the system under agentic workload
- **Prefix Cache Hit %:** Context re-use in agent calls
- **Average Time to First Token (TTFT):** Critical for perceived responsiveness.
- **Average Time Per Output Token (TPOT):** The average time to generate a token post-prefill, indicating decoding speed.
- **Average Prefill & Decode Times:** Detailed breakdown of the LLM's processing stages.

These metrics are vital for understanding and optimizing the underlying LLM performance, directly impacting agent efficiency and cost.

Tracing - Langfuse

01

Framework Native Tracing

Utilized LangFuse for tracing the agents built using the Crew AI framework.

02

Additional Instrumentation

Added instrumentors from OpenInference for Crew AI, LangChain and LiteLLM to additionally capture orchestration flow and raw LLM calls.

03

Unified Trace Context

Wrapped the core crew.kickoff() execution in a Root Span to ensure all actions are part of a single trace.

04

Trace Retrieval

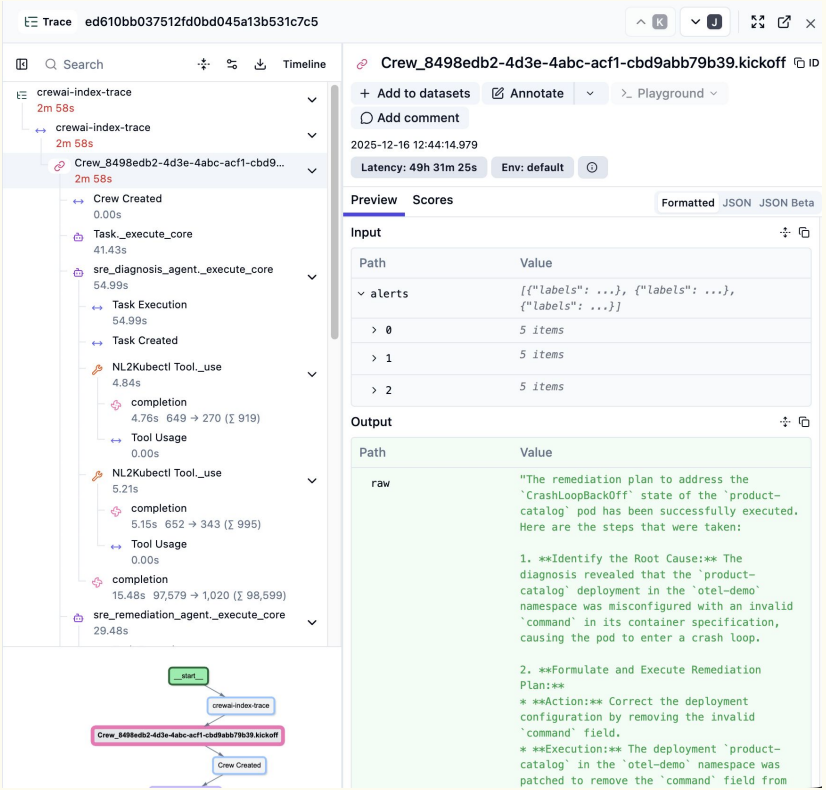
Flushed the trace to the LangFuse dashboard once the agent finishes and fetch it back immediately to use the trace details locally.

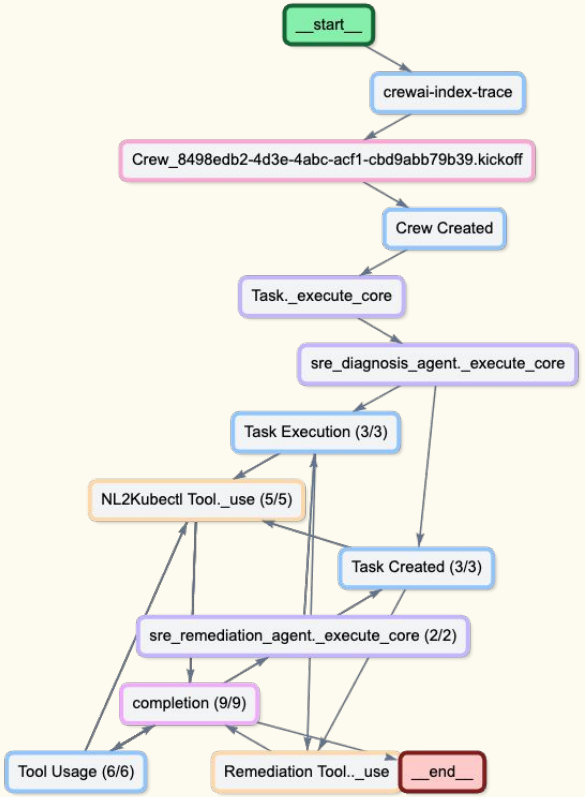
05

Metrics Extraction

Parsed the retrieved trace details and store all steps in the trace in a JSON file for calculating NFRs.

Tracing - Langfuse





Results and Evaluation

ITBench SRE Agent: Gemini 2.5 Pro

	Planning Overhead	End to End Latency (s)	Prompt - Completion Ratio	Number of LLM Calls
Incident 1				
SRE - Plan & Execute	80.71%	1135	42.80	11
SRE - ReAct	92.83%	13126	9.11	21
Mini-SWE	90.40%	218	37.00	10
Incident 16				
SRE - Plan & Execute	85.45%	362	9.775	21
SRE - ReAct	82.75%	401	0.96	16
Mini-SWE	82.20%	161	4.31	10

Results and Evaluation

ITBench SRE Agent: Gemini 2.5 Pro

	Planning Overhead	End to End Latency (s)	Prompt - Completion Ratio	Number of LLM Calls
Incident 30				
SRE - Plan & Execute	80.74%	304	6.14	16
SRE - ReAct	87.95%	793	2.19	37
Mini-SWE	91.82%	283	81.46	10
Incident 102				
SRE - Plan & Execute	81.92%	136	75.53	3
SRE - ReAct	81.69%	358	4.68	14
Mini-SWE	67.69%	296	2.62	7

Results and Evaluation

ITBench CISO React v Plan&Execute Agents: Gemini 2.5 Pro

	Token Usage	Reasoning Tokens	Tool Usage	Throughput(token/s)	Planning Overhead
Incident 1					
React	19,511	3172	3	90.04	49.08%
Plan&Execute	24,234	2011	5	80.22	38.30%
Incident 4					
React	190,613	10,349	18	38.68	6.03%
Plan&Execute	153,891	5,999	13	73.74	32.44%

Across multiple runs and incidents:

- React better for simpler tasks while Plan&Execute is better for complex ones which require long-term planning
- React has variable planning overhead while Plan&Execute is consistent
- Plan&Execute has higher throughput and lower latency

Results and Evaluation

ITBench CISO React v Plan&Execute Agents: Qwen14B-AWQ

	E2E Latency(s)	Token Usage	LLM calls	Max KV Cache Util	Prefix Cache Hit Rate
Incident 1					
React	193.186	116049	3	19.76%	91.47%
Plan&Execute	208.305	120733	5	19.57%	90.32%
Incident 4					
React	81.647	17841	18	8.21%	68.76%
Plan&Execute	75.460	11232	13	6.6%	41.92%

All agents have 0% success rate with Qwen14B

- Even though Incident 1 is easier, more compute and time is spent for resolution
- Plan&Execute has lower prefix cache hit rate indicating agent takes diverse execution paths

Results and Evaluation: Key Takeaways

1. ReAct based agents typically consume more time and makes a higher number of LLM calls due to repeated reasoning loops.
2. Plan&Execute Agents invest effort during early planning and then complete task execution quickly.
3. Higher KV-cache utilization indicates higher memory load, while high prefix cache hit rate indicates repetition of steps
4. While Plan-and-Execute agents show higher prompt-to-completion ratios, it uses less number of tokens for the whole run likely because the generated plan is reused as input for subsequent steps.
5. For SRE, Mini-SWE Agent is generally the fastest with the least number of LLM calls. This is likely due to it not using external tools to run terminal commands unlike the ITBench SRE Agent.

Lessons Learned: Navigating the Complexities of Agent

Survey Depth & Evolving Standards

While ISO/IEC 25010 and 25059 provide a foundational understanding, agentic systems demand an extended NFR taxonomy. Limited prior research on agent-specific NFRs necessitated extensive synthesis across diverse domains to build a comprehensive framework.

Telemetry Standardization Challenge

Frameworks like CrewAI, LangChain, and AutoGPT employ inconsistent telemetry models, hindering unified NFR measurement. While OpenTelemetry's GenAI conventions are emerging, uniform adoption across frameworks is still in early stages.

Agents Succeed via Unintended Paths

Functional success alone is insufficient; agents might bypass safety checks or utilize inefficient shortcuts. This highlights the critical need for **path-aware evaluation** to ensure production readiness and compliance, not just task completion.

Context is King for NFR Thresholds

Generic NFR metrics are ineffective without use-case specific thresholds. For instance, SRE demands sub-5 second latency, whereas FinOps processes may tolerate longer planning times. Tailored evaluation is essential.

Next Steps

01

Expanded Metrics

Add Token Economics & Resource Efficiency metrics

Implement "Hallucination Rate" for RAG-specific tasks to measure faithfulness to documentation

02

Advanced Optimization

Develop auto-tuning frameworks using Reinforcement Learning to optimize agent policies for NFR objectives

03

Continuous Integration of NFRs

Integrate NFR evaluation into the CI/CD pipeline for agents, ensuring ongoing performance and compliance validation.

Collaborative Efforts in Advancing AI Agent Evaluation

Joint Background Research

The group collaborated together to research on existing benchmarks, NFRs, and agentic frameworks.

Weekly Synchronized Meetings

Consistent weekly meetings ensured alignment, facilitated knowledge exchange, and enabled rapid iteration on research findings and technical implementations.

Arjun: Contributed to the survey paper, looked at the integration of NFRs into ITBench, and implemented the Mini-SWE agent using Gemini.
Ayush: Contributed to the survey paper, instrumented CISO, SRE and mini-SWE Agents for NFRs, conducted experiments on SRE and mini-SWE Agents.
Gautam: Literature review, set up and contributed to the survey paper, added agents to benchmark streaming APIs, added performance metrics for langfuse trace
Rishabh: Literature review, maintained codebase and agent environment, vLLM hosting and benchmarking, CISO Agent sandbox script implementation and evaluation, NFR analysis.
Tanmay: Survey paper literature review, initial installation and configuration for ITBench SRE agent environment, implemented granular evaluations for CISO agent reliability.

Thank You

We extend our sincere gratitude to everyone involved in this research project.

Please feel free to reach out with any follow-up questions or discussions.

Mentor

- Ruchi Mahindru, Distinguished Engineer, IBM Research
  Email: rmahindr@us.ibm.com

Contributors

- Arjun: av3315@columbia.edu
- Ayush: ab6106@columbia.edu
- Gautam: gautam.agarwal@columbia.edu
- Rishabh: rj2790@columbia.edu
- Tanmay: ta2832@columbia.edu